

DOCUMENTAÇÃO ESTRUTURA COMPARTILHADA: PROJETO FINAL - MODERNIZAÇÃO DE UM SISTEMA LEGADO

Autor: Matheus Martins da Cruz

1. COMO FOI PROJETADO

A primeira versão de estrutura compartilhada implementada no projeto, foi uma maneira manual onde para cada campo do CopyBook deveria ser analisado e adaptado dentro do C# calculando quantos bytes esse campo iria precisar e em qual byte ele começava e qual terminava, ou seja, por exemplo:

Se no REGCLI.cpy o campo era CLI-ID PIC 9(05). a formatação ficaria, como `.PadLeft(5, '0')` onde o 5 é a quantidade de casas e que deveria preencher com 0 a esquerda do número até a casa 5 ficando, por exemplo: "00001".

Assim fazia para cada campo, o de STRING preenchia com " " espaços a direita, e somava a quantidade total de bytes exigida pelo REGCLI.cpy para funcionar corretamente. Na hora de receber o dados do COBOL precisava receber a string completa e extrair cada substring para cada campo e devolver para o Swagger, por exemplo:

`resultado.SubString(0,5)` para extrair o ID, e `resultado.SubString(5,35)` para extrair o Nome, isso tudo feito de maneira manual. É funcional, mas qualquer alteração no CopyBook, quebraria, necessitando de alterar no código para adaptar.

A segunda versão veio para resolver o problema de ser manual, a ideia é qualquer mudança no CopyBook como adicionar um campo não precisaria refazer os calculos dos bytes de forma manual, alterando poucas coisas para adaptar, aumentando a escalabilidade. Através do CopybookParser que recebe por parâmetro o caminho do CopyBook (no caso desse projeto é o REGCLI que é criado uma cópia na pasta bin para evitar conflitos), ele lê as linhas e, usando comandos de processamento de texto, identifica o que é número (PIC 9) e o que é texto (PIC X), descobrindo automaticamente o tamanho exato para cada campo, ele teria uma função parecida de léxico utilizado em compiladores. E o CopybookWrapper recebe do parser os tamanhos de cada campo e tamanho total do buffer, e na chamada `PayloadCobol()` recebe os dados da requisição e para cada campo ele preenche com sua devida natureza, os numéricos (9) com 0 a esquerda do número e os alfa-numéricos (X) com " " espaços para a direita da string, até preencher o tamanho exato de cada campo. E então invoca a `.dll` passando o buffer com os dados pela memória.

Quando o COBOL retorna os dados é chamada a função ExtrairCampo do wrapper, se o retorno do status for "00" retorna os dados pelo mapeamento do DTO e devolve para o Swagger, caso der erro ele é tratado pelas Exceptions.

2. ESTRUTURA CODIFICADA

- **Parser:**
 - Recebe o CopyBook e preparar ele com o tamanho de cada campo e o tamanho total do buffer para o Wrapper.

Estrutura para a leitura do CopyBook:

```
1  public class Copybook
2      {
3          public string? Nome { get; set; }
4          //"X" String e "9" para numerico
5          public string? Tipo { get; set; }
6          public int Tamanho { get; set; }
7      }
```

Declara as variáveis para o processo:

```
1  public List<Copybook> Campos { get; private set; } = new List<Copybook>();
2  public int TamanhoBuffer { get; private set; } = 0;
3
```

E aplica o método parser utilizando o CopyBook:

```
1 public CopybookParser(string caminhoCpy)
2 {
3     //le as linhas COPYBOOK
4     string[] linhas = File.ReadAllLines(caminhoCpy);
5
6     foreach (string linha in linhas)
7     {
8         //deixa sem os espaços
9         string[] partes = linha.Split(' ', StringSplitOptions.RemoveEmptyEntries);
10
11         //procura o "PIC" no array
12         int indexPic = Array.IndexOf(partes, "PIC");
13
14         //se encontrou "PIC" e existe mais partes
15         if (indexPic != -1 && partes.Length > indexPic + 1)
16         {
17             //antes do PIC é o nome do campo
18             string nomeCampo = partes[indexPic - 1];
19             //depois do PIC é o tipo e tamanho
20             string tipoETamanho = partes[indexPic + 1];
21
22             //primeira letra é o tipo (X ou 9)
23             string tipo = tipoETamanho.Substring(0, 1);
24
25             //verifica o que esta entre os parenteses para pegar o tamanho
26             int startIndex = tipoETamanho.IndexOf('(') + 1;
27             int endIndex = tipoETamanho.IndexOf(')');
28
29             //se encontrou os parenteses e o tamanho é maior que 0
30             if (startIndex > 0 && endIndex > startIndex)
31             {
32                 string tamanhoStr = tipoETamanho.Substring(startIndex, endIndex - startIndex);
33
34                 if (int.TryParse(tamanhoStr, out int tamanho))
35                 {
36                     //adiciona o campo na lista de campos
37                     Campos.Add(new Copybook
38                     {
39                         Nome = nomeCampo,
40                         Tipo = tipo,
41                         Tamanho = tamanho
42                     });
43                     //soma o total do tamanho do registro
44                     TamanhoBuffer += tamanho;
45                 }
46             }
47         }
48     }
49 }
```

- Wrapper:
 - Recebe o buffer do parser e pois métodos para enviar dados para COBOL e receber os dados do COBOL

Declara as variáveis para o processo e o construtor:

```
1 private readonly CopybookParser _parser;
2 public byte[] BufferMemoria { get; internal set; }
3
4 //recebe a leitura do COPYBOOK e cria o buffer de memoria com o tamanho total
5 public CopybookWrapper(CopybookParser parser)
6 {
7     _parser = parser;
8     BufferMemoria = new byte[_parser.TamanhoBuffer];
9 }
```

Método para transformar os dados em que o COBOL aceite

```
1 //converte os dados do C# para os bytes do COBOL
2 public void PayloadCobol(params string[] valores)
3 {
4     StringBuilder payload = new StringBuilder();
5
6     //percorre todos os campos que o Parser mapeou no Copybook
7     for (int i = 0; i < _parser.Campos.Count; i++)
8     {
9         string valorFormatar = i < valores.Length ? valores[i] : "";
10        payload.Append(FormatarCampo(valorFormatar, _parser.Campos[i]));
11    }
12
13    BufferMemoria = Encoding.ASCII.GetBytes(payload.ToString());
14 }
```

Método para Formatar os campos que o Payload precisa

```
1 //função auxiliar para aplicar Zeros ou Espaços
2 private string FormatarCampo(string valor, Copybook campo)
3 {
4     valor = valor ?? ""; //se for nulo vira string vazia
5
6     if (campo.Tipo == "9")
7     {
8         //limita o tamanho da string de entrada para não estourar o buffer caso venha maior
9         if (valor.Length > campo.Tamanho) valor = valor.Substring(0, campo.Tamanho);
10
11         return valor.PadLeft(campo.Tamanho, '0'); //numerico: 00005
12     }
13     else
14     {
15         if (valor.Length > campo.Tamanho) valor = valor.Substring(0, campo.Tamanho);
16
17         return valor.PadRight(campo.Tamanho, ' '); //string: NOME
18     }
19 }
```

Método para extrair os campos que o COBOL enviou de resposta

```
1 //le o array de bytes e extrai os valores
2 public string ExtrairCampo(int indiceCampo)
3 {
4     int offset = 0;
5     //calcula onde o campo começa somando o tamanho dos anteriores
6     for (int i = 0; i < indiceCampo; i++)
7     {
8         offset += _parser.Campos[i].Tamanho;
9     }
10     string textoCompleto = Encoding.ASCII.GetString(BufferMemoria);
11     return textoCompleto.Substring(offset, _parser.Campos[indiceCampo].Tamanho).Trim();
12 }
```

Como funciona dentro do service (esse exemplo é do método consular)

```
1 public ClienteResponseDTO Consular(int id)
2 {
3     var parser = new CopybookParser(_caminhoCopybook);
4
5     var wrapper = new CopybookWrapper(parser);
6
7     wrapper.PayloadCobol(id.ToString(), "", "", "", "");
8
9     DllConfig.cob_init(0, IntPtr.Zero);
10
11     DllConfig.CONSLI(wrapper.BufferMemoria);
12
13     int cliId = Convert.ToInt32(wrapper.ExtrairCampo(0));
14     string cliNome = wrapper.ExtrairCampo(1);
15     string telefone = wrapper.ExtrairCampo(2);
16     string email = wrapper.ExtrairCampo(3);
17     string statusRetorno = wrapper.ExtrairCampo(4);
18
19     if (statusRetorno == "00")
20     {
21         var cliente = ClientesDTOMappings.ToResponse(cliId, cliNome, telefone, email);
22         return cliente;
23     }
24     else if (statusRetorno == "44")
25     {
26         throw new KeyNotFoundException("Cliente não encontrado.");
27     }
28     else
29     {
30         throw new Exception("Erro ao consultar o cliente.");
31     }
32 }
```